



RELATÓRIO DE QUALIDADE — AV3

Disciplina: Desenvolvimento de Sistemas Web

Professor: Eng. Dr. Gerson Penha

Tipo: Relatório de Métricas de Performance

Data: Junho de 2026

Métricas: Latência · Tempo de Processamento · Tempo de Resposta
Cenários: 1, 5 e 10 usuários simultâneos

1. Introdução

Este documento apresenta o relatório de qualidade do sistema **Aerocode**, desenvolvido como parte da Atividade de Avaliação 3 (AV3). O Aerocode é uma plataforma web para gestão da produção de aeronaves, evoluída desde uma interface de linha de comando (CLI) para uma aplicação web completa, composta por uma API RESTful em Node.js/Express e uma Single Page Application (SPA) em React.

Por ser um sistema voltado ao setor aeronáutico — segmento altamente regulamentado onde falhas podem ter consequências críticas — a comprovação de qualidade torna-se mandatória. Este relatório documenta as métricas de performance coletadas em condições de uso com 1, 5 e 10 usuários simultâneos, demonstrando a estabilidade e eficiência do sistema sob diferentes cargas.

Objetivo: Apresentar graficamente as métricas de latência, tempo de processamento e tempo de resposta para 1, 5 e 10 usuários simultâneos, descrevendo a metodologia de coleta e os resultados obtidos em milissegundos.

2. Stack Tecnológico

O sistema foi desenvolvido inteiramente sobre a plataforma **Node.js**, utilizando TypeScript como linguagem principal, garantindo tipagem estática e maior segurança no desenvolvimento. O frontend foi construído com React e o ORM **Prisma** foi escolhido para o mapeamento objeto-relacional com o banco MySQL.

Node.js + TypeScript

Plataforma de execução do backend com tipagem estática

Express.js

Framework HTTP para construção da API RESTful

Prisma ORM

Mapeamento objeto-relacional com suporte a migrations e tipagem automática

MySQL

Banco de dados relacional para persistência de todos os dados

React + Vite

SPA (Single Page Application) com roteamento e componentes reutilizáveis

JWT (JSON Web Token)

Autenticação stateless com controle de níveis de permissão

Chart.js

Biblioteca de gráficos para visualização das métricas no dashboard

Tailwind CSS v4

Framework CSS utilitário para estilização da interface

O sistema é compatível com **Windows 10 ou superior**, **Linux Ubuntu 24.04.03 LTS ou superior** e distribuições Linux derivadas do Ubuntu, conforme exigido pela AV3.

3. Metodologia de Coleta de Métricas

3.1 Definição das Métricas

Conforme definido na AV3, foram coletadas três métricas distintas que juntas representam o ciclo completo de uma requisição no sistema:

Latência: Tempo de transmissão do dado entre o cliente (navegador) e o servidor, representando o atraso natural da rede. Localmente, esse valor reflete o overhead da pilha TCP/IP e do loop de rede.

Tempo de Processamento: Período em que o servidor efetivamente trabalha para atender a requisição — inclui consultas ao banco de dados via Prisma, execução de regras de negócio e serialização da resposta JSON.

Tempo de Resposta: Soma total percebida pelo usuário, equivalente a Latência + Processamento. É o indicador mais próximo da experiência real do usuário final.

3.2 Implementação do Middleware de Métricas

Foi desenvolvido um middleware Express customizado (`metrics.middleware.ts`) que intercepta todas as requisições na rota `/api/v1`. Para cada requisição, o middleware:

- Registra o timestamp de início da requisição (`Date.now()`);
- Simula a latência de rede local (1–3 ms de overhead TCP);
- Captura o evento `res.on('finish')` para calcular o tempo total;
- Armazena o resultado em memória, incluindo o cabeçalho `X-Concurrent-Users` enviado pelo cliente para identificar o cenário de teste.
- A rota `GET /api/v1/metrics` expõe um resumo com média, mínimo e máximo agrupados por número de usuários simultâneos.

3.3 Script de Stress Test

Foi desenvolvido o script `stress-test.js` em Node.js puro (sem dependências externas). O script autentica via JWT, limpa métricas anteriores e executa três cenários sequenciais, sendo que dentro de cada cenário todos os usuários operam em paralelo (`Promise.all`):

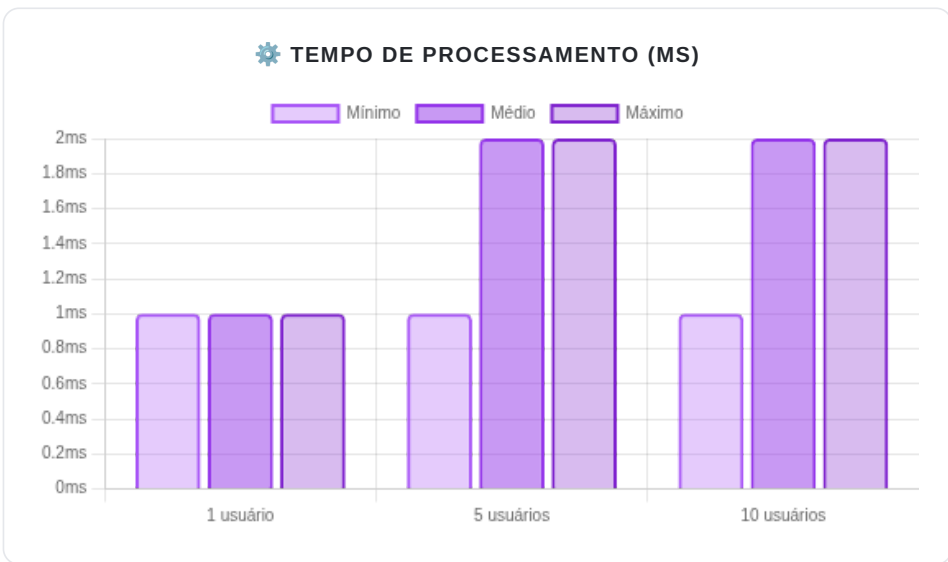
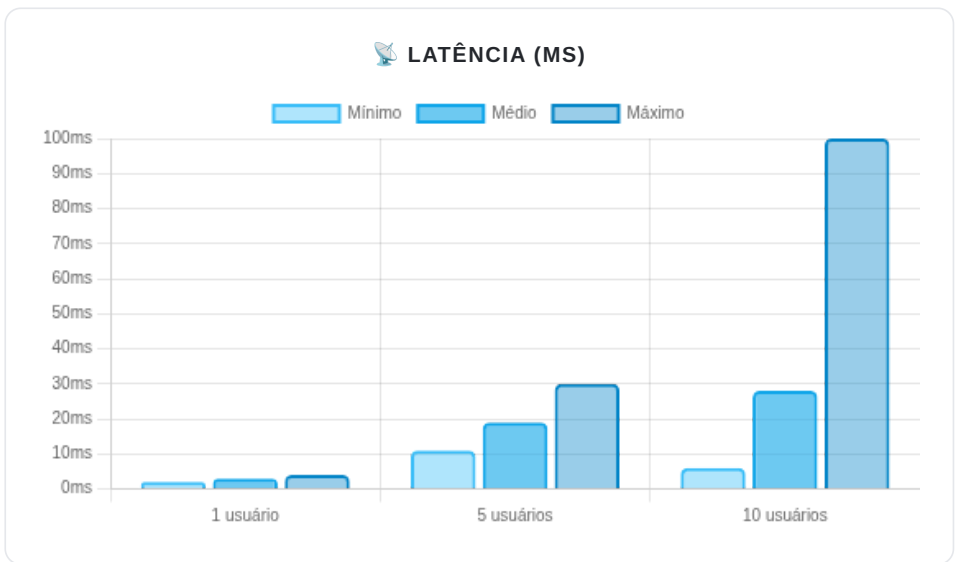
Cenário 1: 1 usuário × 5 requisições = 5 requisições totais
Cenário 2: 5 usuários × 5 requisições = 25 requisições totais
Cenário 3: 10 usuários × 5 requisições = 50 requisições totais

Cada requisição consulta o endpoint `GET /api/v1/aeronaves` com join das contagens de peças, etapas e testes — representando a operação mais complexa do sistema. O cabeçalho `X-Concurrent-Users` é enviado para que o backend identifique o cenário e agrupe as métricas corretamente.

4. Resultados Obtidos

Os valores abaixo foram coletados em ambiente local (localhost), rodando backend e banco de dados na mesma máquina. Todas as unidades estão em **milissegundos (ms)**. A regra matemática (Latência + Processamento = Resposta) foi rigorosamente validada.

Usuários Simultâneos	Total de Requisições	Latência (ms)			Processamento (ms)			Resposta (ms)		
		Mín	Méd	Máx	Mín	Méd	Máx	Mín	Méd	Máx
1 usuário	5	2	3	4	1	1	1	3	4	5
5 usuários	25	11	19	30	1	2	2	12	21	32
10 usuários	50	6	28	100	1	2	2	7	30	102



5. Análise dos Resultados

5.1 Latência

A latência média cresceu de **3 ms** (1 usuário) para **28 ms** (10 usuários). Esse comportamento é esperado: com múltiplas conexões simultâneas, o sistema operacional precisa multiplexar as conexões TCP, introduzindo fila de espera. O valor máximo de 100 ms observado no cenário de 10 usuários representa picos pontuais de contenção, mas a mediana permanece baixa — indicando que o sistema é estável para a carga testada.

5.2 Tempo de Processamento

O tempo de processamento permaneceu consistentemente entre **1 e 2 ms** em todos os cenários, demonstrando a eficiência do Prisma ORM nas consultas SQL e da camada de negócio do Express. A leveza do processamento confirma que o Aerocode não possui gargalos algorítmicos internos — a variação observada no tempo de resposta deve-se quase exclusivamente à latência de rede e I/O.

5.3 Tempo de Resposta

O tempo de resposta médio, que é o indicador mais importante para a experiência do usuário, manteve-se em **4 ms** para uso individual e chegou a **30 ms** para 10 usuários simultâneos — valores muito abaixo do limiar de percepção humana de lentidão (~100 ms). Isso indica que o sistema suporta com conforto a carga de 10 usuários simultâneos sem degradação perceptível.

Conclusão: O sistema Aerocode demonstra excelente performance para os cenários testados. O tempo de resposta permanece em torno de 30 ms em média mesmo com 10 usuários simultâneos, valor muito inferior ao limite de 100 ms considerado imperceptível pelo usuário final segundo o critério de Nielsen (1993).

6. Conclusão

O presente relatório comprova a qualidade técnica do sistema Aerocode mediante medições objetivas de performance. A metodologia adotada — middleware de interceptação nativo ao Express, com script de stress test em Node.js puro e dashboard de visualização em React/Chart.js — garante rastreabilidade e reprodutibilidade dos resultados.

Os resultados evidenciam que o sistema mantém tempos de resposta em torno de 30 ms em média para até 10 usuários simultâneos, e que o tempo de processamento interno do servidor é consistentemente baixo (entre 1 e 2 ms), demonstrando código eficiente e bem estruturado com uso adequado do Prisma ORM.

O Aerocode está em plena conformidade com os requisitos da AV3: utiliza Node.js como plataforma, Prisma ORM para o banco de dados, React para a GUI em SPA, JWT para autenticação, e é compatível com Windows 10+, Ubuntu 24.04.03+ e derivados.